

The Identifiers Are the Specification: Auditing What Neural Decompilation Benchmarks Measure

Raafat Abualazm, Ayman Abo Elhassan, and Amr G. Wassal
Computer Engineering Department, Faculty of Engineering
Cairo University, Giza, Egypt
raafat.202210476@eng-st.cu.edu.eg

Abstract—Neural decompilers are evaluated on benchmarks ported from HumanEval. An audit of six published evaluation configurations finds that where the original identifiers reach the model is decided by the benchmark split rather than the system, and that none of the six reports an identifier-scrubbed control arm. We ask what such scores measure by decomposing the model’s input on a frozen checkpoint, varying one channel at a time and replicating the channel arms across five sampling seeds, extended to six under a pre-registered amendment. Arms outside that family carry fewer seeds and are labelled as such.

The channels separate. On the leakage-clean 174-task set across six sampling seeds, a typed contract supplied at inference with *no training* moves `pass@10` from 4.8 to 10.2 (mean paired difference +5.3) and `compile@10` from 126.0 to 168.2 (+42.2); both effects are positive in every one of the six seed blocks, so an exact paired sign-flip test gives $p=2/2^6 = 0.031$. The arms share seeds, so this blocked test is the valid one, and six blocks is the fewest it can resolve. The other two channels are directional at this precision: deleting recovered constants moves `pass@10` by -2.2 ($p=0.25$) with no compilation effect detected, and permuting which binary belongs to which task moves `compile@10` by -8.7 (sign-consistent, $p=0.031$) with no correctness effect detected ($+0.4$, $p=0.875$). A sixth seed was added under a pre-registered amendment with a single confirmatory test and a hard stop; it is the only inference we claim at $\alpha=0.05$. A pre-registered oracle control that supplies the same typed contract but replaces the binary payload with the empty string — an out-of-distribution input — reaches the highest compilation we measured, 173.5 of 174, while solving no more tasks than baseline.

The dominant channel is the bundle of semantic identifiers. On a frozen checkpoint, replacing the function name, every parameter name and the symbol in the disassembly with opaque equivalents — leaving types, arity and the instruction stream byte-identical — *improves* compilation by 7.5 pp while collapsing `pass@10` from 27.27% to 4.55%, losing 35 tasks and gaining none (exact $p \approx 6 \times 10^{-11}$). Those identifiers function as a compact specification; our design does not separate recall of a memorised solution from inference over a descriptive name, and we claim only the former where the evidence supports it. Independently, an untrained base model prompted with the signature alone, never shown a binary, is not measurably worse than the same model fine-tuned on binaries (-0.0195 , $p=0.549$).

We also establish what this protocol can resolve at all. Re-running one frozen checkpoint under five seeds moves `pass@10` across 2–6 of 174 tasks (SD = 1.79), so differences under about three tasks — including several in this literature and several of our own — are indistinguishable from reseeding the same model. A stronger form of the same defect is that `pass@k` can be reproducible in aggregate while being irreproducible in composition: our strongest arm reports 10.0 ± 1.0 solved tasks

across five seeds, yet those five runs solve 24 distinct tasks and agree on only 4 (mean pairwise Jaccard 0.375). The same runs scored on compilation, which operates an order of magnitude higher, reproduce at 0.973. Against that floor, an exploratory sweep of sixteen architectural interventions and a rejection-sampling programme across six checkpoints yield nothing that survives correction, and a frontier reasoning model with ten attempts solves four tasks.

We argue that a name-scrubbed arm, a seed-replicated noise floor and an input-channel decomposition are together necessary for a decompilation result to be interpretable, and we document an evaluation-leakage defect that invalidated part of our own earlier evidence.

Index Terms—neural decompilation, benchmark validity, negative results, evaluation methodology, reverse engineering, large language models

I. INTRODUCTION

Decompilation recovers readable source from compiled binaries. Traditional decompilers emit correct but unidiomatic output with synthesised identifiers, and a growing literature applies large language models to close that gap [1], reporting functional-correctness rates that look encouraging.

Nearly all of that literature evaluates on benchmarks ported from HumanEval [2]. This paper asks what those scores measure. Our answer, from a controlled decomposition of the model’s input, is that they substantially measure whether the model can identify the task from the human identifiers it is given — which together form a compact natural-language specification — and that the remaining channels divide cleanly between those that produce compilable code and those that produce correct code.

We are deliberate about the mechanism we do *not* claim. A descriptive identifier such as `sortViolationsBySeverity` may be recalled from pretraining or read as a specification and solved afresh; our manipulation removes the identifier and cannot distinguish these. We therefore describe the defect as *specification leakage* and reserve claims of memorisation for where separate evidence supports them.

A. Which evaluations this critique applies to

We surveyed prompt-construction code, released datasets and prompt templates rather than assert where the practice

occurs. The survey covers published neural decompilers and decompilation benchmarks from 2024–2026 for which a prompt template, prompt-construction code or a released evaluation dataset is publicly available; a configuration whose prompt could not be established from an artifact was recorded as unverifiable and excluded rather than inferred. Table I reports the six configurations over four systems that met that bar, each row checked against the artifact independently of the search that located it.

The finding is that **identifier exposure is a property of the benchmark split and the input channel, not of the system**. No verified C-target system supplies a name, parameter names or a signature as a dedicated prompt *field*; LLM4Decompile, SALT4Decompile and SK2Decompile share one single-slot template. But the symbol reaches the model inside the assembly regardless. On HumanEval-Decompile that symbol is the semantically empty `func0` in 656/656 rows, so nothing leaks, and the benchmark should be credited for it. On the *same system*’s GitHub2025 and ExeBench splits the first line of the assembly is the true symbol — `mi_cmp_dynamic_unique:`, `StateIdle:` — and C++ rows carry the demangled signature with parameter types and arity ahead of any instruction. Identical prompt code, opposite exposure. This is also why Section VI scrubs the disassembly symbol and not merely the declaration: that is the channel the field actually leaks through.

Two limits on our own scope follow. First, we could not establish that the practice is prevalent in *modern-language* evaluations. We located no Dart, Java, Kotlin, Swift, Go or Rust neural-decompilation evaluation other than our own prior studies, so a prevalence claim we made in earlier work is withdrawn for want of evidence rather than defended. The one non-C system verified, SCDBench, does supply the original name with parameter types and arity as an explicit prompt field, but its EVM target places selectors in the bytecode by design and it is a single point. Second, what the survey does establish uniformly is the absence of controls: **none of the six configurations reports an identifier-scrubbed or signature-only arm**. The precondition this paper argues for is unmet everywhere we looked, which is the general form of our contribution — not that a particular benchmark is broken, but that none of them can currently tell.

The evidence is uncomfortable in a specific way. When we remove the semantic identifiers and leave everything else — types, arity, the instruction stream, the checkpoint, the compiler, the tests — functional correctness collapses by a factor of six *while compilation improves*. The model does not become worse at writing Dart. It becomes worse at writing the *right* Dart. And in a separate leakage-free study, an untrained baseline that sees only the signature and never sees the binary at all is not measurably worse than any model we trained on the binary.

B. Contributions

- 1) **A benchmark-validity result for neural decompilation** (Section VI): an inference-time ablation on a frozen checkpoint shows that the semantic identifiers, not the binary,

TABLE I
PROMPT CHANNELS IN PUBLISHED NEURAL DECOMPILE
EVALUATIONS. EACH ROW IS GROUNDED IN RELEASED
PROMPT-CONSTRUCTION CODE, A RELEASED DATASET, OR A VERBATIM
PROMPT TEMPLATE. “SIG.” DISTINGUISHES THE *original* SOURCE
SIGNATURE (NEVER SUPPLIED) FROM A DECOMPILER-INFERRED ONE.

system / split	name	params	sig.	ctrl	seeds
L4D-End / HumanEval-Dec.	<code>func0</code>	—	—	no	1
L4D-Ref / HumanEval-Dec.	<code>func0</code>	<code>param_N</code>	Ghidra	no	1
L4D / GitHub2025, ExeBench	original	—	C++ types	no	1
SALT4Dec. / Dec.-Eval, MBPP	<code>func</code>	<code>var1..</code>	—	no	1
SK2Dec. / stripped	<code>sub_XXXX</code>	<code>a1, a2</code>	IDA	no	1
SCDBench / EVM-Solidity	original	—	+types, arity	no	1
<i>totals</i>				0/6	0/4

carry most of the measured correctness, and an untrained signature-only baseline that never sees a binary is not measurably worse than models fine-tuned on binaries.

- 2) **A measured resolution floor for `pass@k`** (Section IV). Re-running one frozen checkpoint under five sampling seeds moves `pass@10` across 2–6 of 174 tasks (SD = 1.79). Most differences reported in this literature — including several of our own — fall below that floor, which we argue makes a seed-replicated floor a precondition for interpreting any `pass@k` comparison.
- 3) **Aggregate reproducibility without compositional reproducibility** (Section V). Across every arm, `pass@k` counts are far more stable than the task sets they summarise: our best arm’s count varies by SD = 1.00 while its five runs agree on only 5 of the 25 tasks they collectively solve. Scoring the *same* runs on compilation, which operates at a much higher rate, reproduces at Jaccard 0.83–0.97, which localises the defect to the low-rate regime rather than to the harness or the benchmark. We give a two-integer reporting fix.
- 4) **An inference-time channel decomposition** (Section X). On a frozen checkpoint, supplying a typed contract moves `compile@10` from 126.0 to 168.4 of 174 with *no training*; removing recovered constants collapses `pass@10` while leaving compilation statistically unchanged; permuting which binary belongs to which task moves compilation and not correctness; and withholding the binary entirely, while supplying the same typed contract, maximises compilation while solving no more than baseline. Supplying the contract at inference and fine-tuning on it both help; on the primary endpoint the training step is the larger of the two, which contradicts a claim we made in earlier work.
- 5) **Exploratory architecture and objective sweeps** (Section VII): sixteen controlled interventions with one significant effect and a control-flow-edge effect whose sign reverses across seeds, together with a rejection-sampling programme closed across six checkpoints and three arrangements.
- 6) **A frontier ceiling on name-scrubbed data** (Section IX), measured under a sealed, hash-pinned harness and reported as a single-provider, single-date reference point.

7) **A documented evaluation-leakage post-mortem** (Section XI) and the protocol that replaced it (Section XII).

C. *Relationship to our prior work*

This paper builds on two earlier studies by the authors. A SANER ERA paper introduced the Dart decompilation task and a first fine-tuned pipeline [3], and a journal study analysed fine-tuning effectiveness and metric validity on that pipeline [4]. Both report decompilation performance *using* a benchmark that supplies the function signature. The present paper is the first to ask whether that benchmark is measuring decompilation at all, and it answers in the negative. Section VIII summarises results from [4] as context and claims no novelty for them; Sections IV, VI, X, VII, IX and XI are new.

II. BACKGROUND AND RELATED WORK

Classical decompilation descends from Cifuentes [5] through structural analysis [6], control-flow structuring [7] and platforms such as BAP [8]. Neural approaches began with sequence transduction [9], [10] and now centre on LLMs fine-tuned on assembly–source pairs [1].

The current frontier adds structure or iteration atop a language model: control-flow augmentation [11], hierarchical graph abstraction [12], abstract logic trees [13], skeleton-to-skin generation [14], hierarchical attention over assembly [15], feedback-driven refinement [16], and joint code and type prediction [17]. Our results engage this line directly and mostly negatively: Section VII reports a leakage-free null for explicit control-flow and dataflow conditioning, and Section VI shows that the semantic identifiers — which reach the model through the channels catalogued in Table I — account for most of the measured success.

Evaluation has converged on CODEBLEU [18], `compile@k`, and `pass@k` [2], with benchmarks ported in the style of MultiPL-E [19]. Recent work questions whether such benchmarks reflect real-world decompilation [20]; we supply a mechanism for why they do not.

Dart is a demanding and under-studied target. Its AOT compiler performs whole-program optimisation, erases names, and stores literals in a per-isolate object pool addressed through a dedicated register, so constants do not appear inline in the disassembly. Existing Dart tooling recovers structure but not source [21]–[23].

III. EXPERIMENTAL INFRASTRUCTURE

This paper uses five corpora, and results are not comparable across them. Table II states which supports which claim.

HE-154 is 154 of HumanEval’s 164 problems with executable harnesses (median 7 assertions). Two distinct defect populations affect it, and we separate them because they have been conflated. First, 12 of the 154 supplied reference solutions fail their own harness. A failing reference does not make its task unsolvable — another implementation may pass — but it means those 12 tasks cannot be certified by their own gold code, and we treat 142 as the reference-certified set. Three tasks fall in both defect populations, so the strict union leaves 141. Second,

TABLE II
CORPORA. COUNTS ARE NEVER COMPARED ACROSS ROWS.

ID	origin	tasks	supports
HE-154	HumanEval ported to Dart	154	\$VI, \$VII
MF-175	internal: LLM-gen + legacy	175	\$IV, \$V, \$X, \$IX
CB-73	Dart, held out	73	CODEBLEU (prior work)
CP-126	Dart, held out	126	<code>compile@k</code> (prior work)
FL-343	Flutter ARM64 release APKs	343	\$X-G

and independently, 4 tasks carry hidden-test defects that make them unscorable under *any* opaque contract: tests binding an undefined wrapper (#121, #153), tests passing record arguments to a `List<int>` contract (#127), and an unintended string interpolation in the test source (#161). Those 4 are enumerated in a manifest frozen independently of any treatment outcome; excluding them gives the **valid-150** confirmatory set reported alongside the full 154 in Section VI. HE-154 supports the identifier ablation only.

MF-175 is an **internally constructed, held-out x86-64 evaluation suite**, not an external benchmark. It comprises 136 LLM-generated Dart tasks with supplied test harnesses and 39 legacy standalone Dart SFT programs with generated differential tests. Following signature neutralisation and a deterministic semantic-component split, eligible tasks were individually compiled to source-only AOT ELF’s with Dart 3.12.2 and represented using their extracted target-function F2. The surviving manifests identify 1,580 training tasks and 175 evaluation tasks; a separate expanded file of 2,776 *training rows* was derived from the training split and is not a task pool. MF-175 supports the reseeding, stability, channel-decomposition and frontier analyses.

Two limitations follow, and both bear on how far its results generalise. The retained records for the 39 legacy tasks do not identify their original repositories or licences; we therefore describe them only as legacy internal `dart_all` tasks and make no project-level or real-world provenance claim. More consequentially, *a majority of the suite is model-generated*. Synthetic Dart may lie closer to a decompiler’s pretraining distribution than human-written code, and its identifiers may be more uniformly descriptive than those in production software — either of which could inflate the identifier-channel effects we measure. MF-175 therefore bounds what these channels carry on this construction; it is not a sample of real-world decompilation targets. Each task pairs a reference implementation with an executable acceptance suite, and the references pass their own harness 175/175 — the Rank-0 control of Section IV. One held-out task shares byte-identical acceptance tests with a training row. That task is not inert: it compiles in every arm and every seed, and is solved in every baseline, typed-contract and structure-permuted seed, while no constants-stripped seed solves it. It is excluded from all confirmatory analysis, and counts on MF-175 are reported on the leakage-clean **174-task** subset. Its behaviour is itself a contamination datapoint: a training row is recovered whenever the identifier and constant channels are intact, and lost when the constants are removed.

TABLE III

STUDY MATRIX. TWO DISTINCT SYSTEMS ARE USED; NO RESULT CROSSES THEM.

	identifier (§VI)	channel/floor/stab.	frontier
corpus	HE-154 / valid-150	MF-174	MF (full 175)
model	Qwen3-8B+graph enc.	T5Gemma-2-4b-4b	DeepSeek-V4-Pro
weights	e8e872...	opt-348 83d815...	provider API
seeds	42 (single)	42–46	one complete run
k / samples	10	10	10
Dart	3.11.5	3.12.2	3.12.2 (scoring)
prompt	antigravity-v2/v3	sealed F2 contract	sealed F2 prompt

CB-73 and CP-126 are the disjoint corpora used by the prior work summarised in Section VIII.

Functional comparisons are paired at the task level: McNemar’s exact two-sided test on discordant tasks, with 10,000-iteration bootstrap CIs. Across-seed arm comparisons are **blocked, not independent**: every arm runs on the same five seeds over the same tasks, so the valid randomisation is a paired sign-flip over five seed blocks ($2^5 = 32$ arrangements), not a relabelling of ten free observations. Its smallest attainable two-sided p is $2/32 = 0.0625$, so **no contrast in this study can reach $\alpha=0.05$ whatever its magnitude**; six seeds would be the minimum for significance to be *attainable*, not guaranteed. We report the paired p throughout and treat 0.0625 as the design’s floor rather than as evidence. Rates carry Wilson intervals. $\text{pass}@k$ for $k > 1$ is any-of- k at the task level. $\text{pass}@1$ on MF-175 is the success of the first sampled candidate, not the c/n estimator; it is reported for completeness and carries no claim.

What $\text{compile}@k$ measures, precisely. We state this because several of our results are otherwise easy to over-read. A candidate is concatenated with the task’s real test file and executed with `dart run`; a non-zero exit counts as a compile failure *only* when Dart emits a static front-end diagnostic, while runtime exceptions and failed assertions count as compiled-but-not-passing. Because the tests invoke the function by name with typed arguments and consume its return value, and because Dart is statically typed, passing the front end requires the emitted function’s name, arity and parameter and return types to conform to the call sites, and requires the body to type-check. $\text{compile}@k$ is therefore an *interface-conformance and type-consistency* metric. It is not a test of whether the output is well-formed Dart, and it is not a proxy for correctness. Any intervention that supplies the declared signature is supplying the model with the answer to much of what this metric asks, and we read the compilation column accordingly throughout.

The identifier study and the channel study run on *different systems* (Table III); no comparison in this paper crosses that boundary, and agreement between them is convergence of two instruments, not replication on one.

IV. RESEEDING VARIABILITY UNDER THIS PROTOCOL

Every comparison in this paper, and every comparison in the literature it critiques, is a difference between two $\text{pass}@k$ counts on a benchmark of a few hundred tasks. Before reporting any such difference we measured what this protocol can resolve.

TABLE IV

ONE CHECKPOINT, THE LEAKAGE-CLEAN 174 TASKS, FIVE SAMPLING SEEDS. ONLY THE SEED VARIES.

metric	seed					mean	SD
	42	43	44	45	46		
$\text{pass}@10$	6	6	6	4	2	4.8	1.79
$\text{compile}@10$	123	124	134	125	124	126.0	4.53
$\text{pass}@1$	2	1	2	1	2	1.6	0.55

We re-ran a *single frozen checkpoint* on the same 175 held-out tasks under five sampling seeds, changing nothing else — same weights, same task order, same decoding parameters, same scorer, same evaluation file by SHA-256.

$\text{pass}@10$ ranges from 2 to 6 on an identical model (Table IV). A pre-registered interpretation gate — fixed before the seeds were drawn, and triggered by the observed spread — declares differences of this magnitude uninterpretable under this protocol. The practical consequence is that **a difference of fewer than about three tasks cannot be resolved here**, and we report every such difference as lying within the floor.

Two controls establish that this is sampling variance rather than a broken instrument. First, scoring the reference implementations through the same evaluator returns 175/175 on compilation, $\text{pass}@1$ and $\text{pass}@k$: the harness registers a perfect result when one exists, so null outcomes are not an artifact of measurement. Second, re-deriving the same arms under a later revision of the extraction and scoring code reproduces every count exactly.

We report this as a contribution rather than a caveat, and we are careful about whose results it constrains. The C-target literature surveyed in Section I-A does not report $\text{pass}@k$ at all: its metric is re-executability, or RC/RE/TCP, on a *single greedy decode*, so a decode-seed floor does not transfer to those numbers directly. What does transfer is the absence of any error bar — none of the four papers reports seeds, repetitions, variance or confidence intervals — and the consequence surfaces in their ablations, where component contributions of 0.1–0.3 pp are reported and two are sign-inverted, the ablated model beating the full one. Our floor bounds what $\text{pass}@k$ comparisons can resolve on corpora of this size, our own included; for single-sample metrics the corresponding quantity is simply unreported. Section XII makes a seed-replicated floor a reporting requirement.

V. A REPRODUCIBLE NUMBER OVER AN IRREPRODUCIBLE SET

The floor of Section IV is a statement about the *count*. A count can be stable while the set it summarises is not, and on this corpus it is. For every arm of the channel study we recovered the identity of the solved tasks in each of its five seeds and compared the sets pairwise (Table V).

The count is the stable part. The typed-contract arm reports 10.0 ± 1.00 solved tasks — the tightest count in the study. Yet its five runs solve 24 distinct tasks between them and agree on 4. A single run of that arm therefore understates what the

TABLE V

SOLVED-SET STABILITY ACROSS FIVE SAMPLING SEEDS. ONLY THE SEED VARIES. “ALL 5” IS THE NUMBER OF TASKS SOLVED IN EVERY SEED, “ANY” THE NUMBER SOLVED IN AT LEAST ONE; JACCARD IS THE MEAN OVER ALL TEN SEED PAIRS. COUNTS REPRODUCE FAR BETTER THAN THE SETS THEY SUMMARISE, AND COMPILATION REPRODUCES FAR BETTER THAN CORRECTNESS.

arm	pass@10					compile@10		
	mean	all	any	<i>J</i>	<i>J</i> /chance	<i>J</i>	<i>J</i> /chance	
— constants	2.6	1	7	0.342	45.7	0.845	1.52	
baseline	4.8	1	15	0.203	14.7	0.854	1.50	
structure permuted	5.2	1	12	0.302	20.4	0.830	1.62	
+ typed contract	10.0	4	24	0.375	13.1	0.973	1.04	

configuration can reach by more than half, and overstates what it reaches *reliably* by more than double, while reporting a number that replicates to one task. The baseline is worse: two tasks are solved in all five seeds, fifteen in at least one, and most of that union is solved by exactly one run.

The harness is not the cause. The identical runs scored on compilation put only 3% of the compiling union in a single seed, against 56–71% of the solved union. Same generations, same seeds, same scorer, same tasks: whatever scatters the solved set leaves the compiling set almost fixed, which excludes nondeterminism in generation, in the compiler, or in evaluation order.

But the raw Jaccard gap is not the measure of it. Jaccard rises mechanically with set size — two sets covering 97% of the tasks must overlap heavily — so the 0.30 versus 0.97 contrast is largely an artifact of the rate, not evidence about it. Against the overlap expected for independent random subsets of the same sizes, the ordering *reverses*: solved sets run 13–46× chance while compiling sets run only 1.0–1.6×. There is real task-level structure in what gets solved; the stable core is simply small in absolute terms. The defensible statement is the absolute one — of the 24 tasks the strongest arm ever solves, 4 are solved every time — and it is the absolute figure that governs whether a task-level claim is safe.

This is the empirical form of the unmeasurable left tail: tasks whose per-sample success probability is small relative to $1/k$ are individually indistinguishable from unsolvable, and which of them surface is decided by the draw.

What this costs the literature. It does not threaten aggregate comparisons: the between-arm gaps in Table IX are several times the count SD and survive. It does threaten every *task-level* claim, and those are common — that a method fixes the harder cases, repairs a particular failure mode, or unlocks a specific class of function. On this corpus, two runs of one configuration disagree about roughly 70% of what either solves. No such claim is supportable at these rates without showing that the solved set itself replicates.

The fix is cheap. Alongside the mean, report the number of tasks solved in *every* seed and in *any* seed. Two integers make the gap visible, and their ratio is a direct estimate of how much of a reported score is a stable capability rather than a draw.

TABLE VI

IDENTIFIER ABLATION ON A FROZEN CHECKPOINT, SEED 42. THE FUNCTION NAME, ALL PARAMETER NAMES AND THE DISASSEMBLY SYMBOL CHANGE TOGETHER; TYPES, ARITY AND THE INSTRUCTION STREAM ARE HELD BYTE-IDENTICAL.

Signature shown	p@1	p@10	c@1	compiled
Semantic name	17.66	27.27	77.08	1,187/1,540
Typed, opaque fn0	0.71	4.55	84.61	1,303/1,540
Name only	0.00	0.00	3.18	49/1,540

VI. THE BENCHMARK MEASURES ITS OWN IDENTIFIERS

A. Design A: removing the semantic identifiers

We freeze a checkpoint and hold the HE-154 tasks, the scoring stack and the pinned compiler fixed, varying only the human-readable identifiers: the original semantic HumanEval signature; *neutral_exact*, which preserves types and arity but replaces the function name with an opaque *fn0* and every parameter name with a positional letter; and *name_only*, which gives *fn0* with no types.

We state the manipulation precisely, because it is broader than a function rename and the breadth bounds what may be concluded. In all 154 tasks *neutral_exact* replaces the function name, all 200 parameter-name slots, the symbol in the disassembly headers and branch-target annotations, 15 helper symbols, and the source path. What it leaves untouched is equally exact: parameter types and arity are identical in 154/154, and after masking symbol annotations the instruction stream — opcodes, operands, addresses — is byte-identical in 154/154. The arm therefore removes *every human-meaningful identifier reachable by the model* while holding the binary evidence constant. Effects below are attributable to that bundle, not to the function name alone.

Table VI separates two contributions that the literature conflates, including work predicting code and types jointly [17].

Types and arity carry compilation. Given an opaque typed contract the model emits a function named *fn0* with the requested arity in 1,537 of 1,540 samples, and 1,303 of those compile against the tests — *better* than the named comparator by 7.5 pp. Syntactic competence does not degrade.

The identifiers carry functional correctness. Removing them collapses *pass@10* from 27.27% to 4.55%. Of the comparator’s 42 solved tasks, *neutral_exact* retains 7 — zero gained, 35 lost. Roughly 83% of “solved” tasks were solvable only with the semantic identifiers present. That the floor is reachable rules out a broken harness: 151 of 154 neutral references compile and 142 pass. The scrubbed generations are not broken Dart; they are *semantically wrong* Dart.

The result is invariant to the benchmark’s defects. Repeating the comparison on the valid-150 set of Section III, which drops the four contract-defective tasks, gives 28.0% versus 4.67% — and *the same* 35 losses, 0 gains and 7 retained. The excluded tasks all sit among the 112 that neither arm solves, so removing them moves the denominator and nothing else. The reference-certified 142 cannot be recomputed without per-task rescoring, but it is bounded: even if all 12 reference-failing

TABLE VII

EVERY ARM AGAINST A BASELINE THAT SEES ONLY THE SIGNATURE AND NEVER THE BINARY. PAIRED TASK BOOTSTRAP; McNemar EXACT.

Arm vs. signature-only	$\Delta p@10$	95% CI	p
Trained on raw assembly	-0.0195	[-0.065, +0.026]	0.549
Graph prefix, no edges	+0.0195	[-0.020, +0.058]	0.549
Graph prefix + CFG	+0.0260	[-0.013, +0.071]	0.388

tasks lay among the 35 losses, the containment would remain at least 23 losses to 0 gains. We report all denominators for this reason: the effect does not depend on which validity criterion a reader prefers.

We locate this figure against the standard of Section V rather than leave it to be questioned. The 83% is a ratio of two counts, 42 and 7, and the count is the quantity Section V finds stable; the collapse is a factor of six, far outside any floor. The containment it rests on — 35 losses and *zero* gains — is a task-level statement, but it is the opposite of the fragile case: the discordance is unanimous across all 35 pairs (exact two-sided $p \approx 6 \times 10^{-11}$), at a 27.27% success rate where Table V shows solved sets to be far more stable than in the low-rate arms. The constraint of Section V applies to task-level claims resting on few discordant pairs at low success rates, which this is not. What does remain conditional on this seed is the *identity* of the seven retained tasks, and we make no claim about which they are.

B. Design B: an untrained signature-only baseline

Design A varies a prompt on a fixed model. The complementary question is whether *training* on binaries buys anything over a policy that never sees one. In the leakage-free study of Section VII we compared three binary-consuming arms against a signature-only baseline that is the **untrained base model**: no fine-tuning, no adapter, prompted with the signature alone. It reaches $\text{pass}@10$ of 0.234 (36/154).

No arm is distinguishable from the signature-only baseline. A model fine-tuned on raw assembly is nominally *worse* than an untrained model that never sees the binary; on $\text{pass}@1$ that deficit’s interval excludes zero. The graph-conditioned arms are nominally better by two to three points, with intervals crossing zero.

We are explicit about what this comparison is and is not. The baseline differs from the other arms in *two* respects — it is untrained and it is shown a different input — so it does not isolate the binary channel and cannot support a decomposition. What it does support is a bound, and the bound is the uncomfortable part: several hundred GPU-hours of fine-tuning on binaries did not produce a policy measurably better than the base model reading a signature. Designs A and B are methodologically independent, and both are consistent with the benchmark being insensitive to binary evidence; neither establishes that the binary is uninformative, and we do not claim it.

TABLE VIII

ARCHITECTURAL INTERVENTIONS (SEED 42, PAIRED TASK BOOTSTRAP). EXACTLY ONE IS SIGNIFICANT, AND IT REMOVES THE CHANNEL RATHER THAN RESTRUCTURING IT.

Intervention	$\Delta p@10$	p
No-GINE vs. no-edge GINE	+0.0065	1.000
No-GINE vs. CFG+DFG GINE	+0.0260	0.219
CLAP-ASM vs. GraphCodeBERT encoder	-0.0065	1.000
Four block vectors vs. one CLS	+0.0000	1.000
Eight block vectors vs. one CLS	-0.0130	0.688
Hierarchical regions vs. block prefix	-0.0195	0.375
Region maximum 16 vs. no regions	+0.0130	0.688
No global attention	-0.0130	0.625
No block positions	-0.0195	0.375
Two- vs. four-layer GINE	+0.0065	1.000
Cross-task permuted graph	-0.0130	0.500
Shuffled block order	-0.0065	1.000

Zero the prefix gate -0.0455 0.016

Three further contrasts from the same sealed sweep are omitted for space; all are null (-0.0065, $p=1.000$; +0.0000, $p=1.000$; -0.0195, $p=0.508$). Including them, 16 contrasts were run over 15 intervention arms, and the single nominally significant result below is the only one among all 16.

VII. EXPLORATORY: ARCHITECTURE AND OBJECTIVE INTERVENTIONS

A natural response is that the architecture was inadequate. We ran a pre-registered sweep with freeze rules fixed in advance and a reviewer-objection matrix mapping each plausible criticism to a control, rather than searching until a configuration won.

A three-seed core ablation (prefix with no edges, +CFG, +CFG+DFG; seeds 42–44) puts the no-edge arm at the best mean $\text{pass}@10$ (.260) with every edge effect’s hierarchical-bootstrap interval crossing zero. The instructive part is the instability: at seed 44, CFG versus no-edge is 1 gain / 8 losses (exact $p=0.0391$) while adding DFG to CFG is 7 gains / 0 losses ($p=0.0156$) — two nominally significant effects in opposite directions within one seed. A direction that reverses across seeds is not evidence of a null; it is no evidence of a reproducible effect either way.

Table VIII reports the component isolations. Encoder identity, message-passing depth, prefix width, hierarchical regions, global attention and positional encodings all leave $\text{pass}@10$ statistically unchanged.

Multiplicity. We report the correction rather than wait to be asked. With 16 contrasts, a Holm threshold at the smallest rank is ≈ 0.003 , and the prefix-gate effect at $p=0.016$ does not survive it; under the global null, seeing at least one $p < 0.05$ among 16 tests has probability ≈ 0.56 . We retain the arm not as an independently significant finding but because it was predicted in advance, its discordance is unanimous (0 gains, 7 losses), and it converges with Designs A and B. Read alone, it is consistent with chance.

The one nominally significant cell does not survive correction. Zeroing the prefix gate — deleting the binary channel outright — costs 0.0455 with 0 gains and 7 losses ($p=0.0156$), and freezing the encoder drops $\text{compile}@1$ from

0.78 to 0.48. Meanwhile substituting *another task’s* graph costs only 0.0130 ($p=0.50$) and shuffling block order costs 0.0065 ($p=1.00$). The decoder uses the channel, but is largely indifferent to what it contains: the learned assembly-to-prefix compression is load-bearing, the explicit topology is not.

A. Disclosure

This sweep is exploratory, and we treat it as such: the core ablation has three of five planned seeds, the component isolations are single-seed diagnostics, and the study’s own provenance audit header reads FAIL across its 38 runs (10 showing an aggregate/candidate-replay disagreement, resolved by treating candidate-level replay as canonical). None of its conclusions is confirmatory; the certified result of this paper is the pre-registered channel test of Section X-C, not anything in this section.

VIII. PRIOR FINDINGS ON THIS CORPUS

We cite [4] as context only, claim no novelty for it, and — because it remains under journal review — do not reproduce its results here. In summary, it reports fine-tuning configurations without functional-correctness gains and a divergence between surface metrics and correctness on this corpus. Those findings establish that training on this corpus does not improve functional correctness and that surface metrics do not track it. They do not address a prior question: what the benchmark measures when it does report a score. That question is this paper’s subject.

IX. A FRONTIER REFERENCE POINT ON IDENTIFIER-SCRUBBED DATA

Our first frontier measurement reported 5/60. An internal audit ruled it indefensible — assembly truncated at 12,000 characters so 33 of 60 prompts lost the function tail, anonymous closures omitted by the symbol regex, a cache that was not content-addressed, and unknown effective k with silent failures. We never report it as a result.

The replacement harness pins a paired manifest over 175 ordered tasks and enforces eight invariants, all verified: `provider_prompts_exclude_gold_and_tests,`
`no_prompt_truncation,` `measure_only_seals_verified,`
`prompt_artifact_hashes_verified,`
`same_acceptance_tests,` `same_f2_grammar,`
`same_ordered_175_tasks,` `same_prompt_budget_contract.`
 Evaluator, Dart binary, tokenizer and prompt artifacts are SHA-256 pinned, each run carries a 256-bit completion attestation, and the harness refuses to emit a score for an incomplete run.

On the complete run — 175/175 tasks, 1,750/1,750 slots — DeepSeek-V4-Pro at $k=10$ solves **4 tasks**, `pass@10` 2.29%. We separate the two denominators explicitly, because conflating them is easy and we have done it: at the *task* level `compile@10` is $86/175 = 49.1\%$, while at the *candidate* level $300/1,750 = 17.1\%$ of generations `compile` and $11/1,750 = 0.63\%$ `pass`. Every slot carries `finish_reason: stop`, so the score is not a truncation

artifact. Our 8B student reaches `compile@10` of 69.1% on the same tasks: whatever the frontier model contributes here, it is not syntactic competence. This is one provider’s model on one date under one decoding configuration; it is a reference point, not a ceiling. No score exists for the other frontier models we queried — they reached preflight or quota only, and absence of a score is not a score of zero.

X. THE INFORMATION CHANNEL

Sections VI and VII establish that the name carries the score and that restructuring the model does not. This section asks the constructive question: *which* channel carries recoverable signal. We decompose the input on a frozen checkpoint, varying one channel at a time.

One chronological fact is disclosed rather than left in the artifact: seed 42 predates the other four — its baseline is the sealed epoch-ablation run and it served as diagnostic context while the interventions were designed — whereas seeds 43–46 are fresh replications. Excluding seed 42 changes no direction in any contrast. Each intervention was replicated across five sampling seeds (42–46); the typed contract, baseline and structure-permutation arms carry a sixth (47) added under the pre-registered amendment of Section X-C. Table IX gives the decomposition; every effect is tested by an exact paired sign-flip over the seed blocks, the valid randomisation given that all arms share the same seeds.

What the six-seed test establishes. With five blocks the paired floor was $2/2^5 = 0.0625$, so no contrast could reach $\alpha=0.05$ whatever its magnitude. We therefore pre-registered one sixth seed with a *single* confirmatory test — the typed contract against baseline on `pass@10` — and a hard stop (Section X-C). Seed 47’s difference is +6 tasks, the sixth positive block of six, giving $p=2/2^6 = 0.031$: the typed-contract effect on functional correctness is significant. Its compilation effect is sign-consistent over the same six blocks at the same p , reported as a secondary descriptive rather than a second confirmatory test. Structure permutation is sign-consistent on compilation across six blocks (−8.7 tasks) and null on correctness; removing constants stays at −2.2 tasks ($p=0.25$), directional and below resolution. The paper’s own standard — replicate before interpreting — is now met for the one effect it certifies.

A. Types, supplied at inference, without training

Replacing the input’s signature with a typed but opaque contract — `fn0` with declared return and parameter types, parameter names reduced to `p0, p1, ...` — and changing nothing else raises `compile@10` from 126.0 ± 4.53 to 168.4 ± 2.19 tasks of 174 (+42.4) and `pass@10` from 4.8 ± 1.79 to 10.0 ± 1.00 (+5.2); both reach complete separation across the five seeds, so both return the paired floor of 0.0625. On the paired seed-42 comparison the compilation gain is 49 tasks against 3 losses; at candidate level compilation rises from 578/1,740 to 1,323/1,740, and diagnostics attributable to wrong arity or wrong types fall from 1,081 to 312.

TABLE IX

INPUT-CHANNEL DECOMPOSITION ON A FROZEN CHECKPOINT, LEAKAGE-CLEAN 174 TASKS, SIX SAMPLING SEEDS FOR THE CONFIRMATORY ARMS AND FIVE ELSEWHERE, AS MARKED. EACH CHANNEL HAS A DISTINCT SIGNATURE, AND TWO OF THE SIX TESTED CELLS SHOW NO DETECTED EFFECT.

input	pass@10	compile@10
baseline	5.0 ± 1.67	125.8 ± 4.07
+ typed contract	10.3 ± 1.21	168.0 ± 2.19
– constants [‡]	2.6 ± 1.14	124.2 ± 4.66
structure permuted	5.0 ± 1.79	117.2 ± 2.93
typed contract, no binary [†]	4.0 (5, 3)	173.5 (174, 173)
<i>effect vs. baseline (exact paired sign-flip, two-sided; six-seed floor $2/2^6 = 0.031$)</i>		
+ typed contract	+5.3, $p=0.031$	+42.2, $p=0.031$
– constants [‡]	–2.2, $p=0.25$	–1.8, $p=0.50$
structure permuted	0.0, $p=1.00$	–8.7, $p=0.031$

Baseline, typed contract and structure permutation are six seeds (42–47); [‡]constants is five (42–46), its status unchanged by a sixth block. [†]Two seeds only, per-seed counts in parentheses: a gold-derived oracle control with an out-of-distribution empty input, reported without a permutation effect and without any equivalence claim (Section X-D).

We note that the functional gain is visible only once seeds are replicated. At a single seed the difference is $6 \rightarrow 9$ tasks, inside the floor of Section IV and not interpretable; across five seeds it is a 5.2-task effect at the paired floor of $p=0.0625$. This is the concrete case for the reporting requirement of Section XII.

The comparison that matters is what training adds on top, and here we correct a claim we have made in earlier work. A model *fine-tuned* on the same typed contract reaches `compile@10` of 171 against 170 — one task — which we previously reported as showing that essentially the whole attainable gain is information rather than optimisation. That reading selected the secondary metric. On the primary endpoint at the same seed, `pass@10` goes from 10 to 14. The full triple in functional terms is $6 \rightarrow 9 \rightarrow 14$: supplying the contract adds three tasks, training on it adds five. Neither step is individually resolvable (exact McNemar $p=0.27$ for the training step), and the honest summary is that information and optimisation contribute comparably here, with the data unable to separate them.

B. A three-way dissociation

The remaining two interventions do not simply produce smaller versions of the same effect. Each moves a different metric, and each leaves the other unchanged.

Constants: correctness moves; no compilation effect detected. Deleting recovered string and numeric literals, with the structural payload byte-identical, reduces `pass@10` from 4.8 ± 1.79 to 2.6 ± 1.14 (-2.2 , $p = 0.25$) while `compile@10` shows no detected difference at 124.2 ± 4.66 (-1.8 , $p = 0.50$). On the 96 tasks that actually carry literals the seed-42 count falls from 4 solved to 0. As a control, the 79 tasks containing no literals return predictions *byte-identical* to baseline, confirming that the intervention is surgical and the harness deterministic.

Stripped of constants the model still writes Dart that compiles; it stops writing Dart that is correct.

Structure: compilation moves; no correctness effect detected. An exact derangement, in which each task receives another task’s assembly and control-flow graph while keeping its own constants, leaves `pass@10` statistically indistinguishable from baseline (5.2 ± 1.92 versus 4.8 ± 1.79 ; $+0.4$, $p = 0.875$) while reducing `compile@10` to 117.4 ± 3.21 (-8.6 , $p = 0.0625$). Running the model on an entirely different function’s binary changes functional correctness by $+0.4$ tasks — a nominal *gain*, well inside the floor.

We note what this arm does *not* show. The derangement preserves each task’s own recovered constants and replaces only the structural payload, so it isolates the identity of the control-flow evidence rather than removing the binary channel. Taken with the constants ablation above, the two arms together explain each other: permuting structure is nearly free *because* the correctness-carrying channel is left intact.

We label this an *exploratory channel pattern*, not a dissociation. Across five paired seeds the three contrasts point in different directions on the two metrics, but the design establishes no contrast at $\alpha=0.05$ and non-significance in the remaining cells is not equivalence. The defensible summary is that the typed contract moves both metrics by large margins, removing constants moves correctness by -2.2 tasks with no compilation effect detected, and permuting structure moves compilation by -8.6 tasks with no correctness effect detected — all of it directional evidence awaiting a six-seed test.

Across all four arms and every seed, the mean number of distinct programs emitted per task lies between 9.98 and 10.00 of 10, so none of these effects is attributable to degenerate decoding.

C. The pre-registered sixth seed

The five-seed design cannot certify any effect: with all arms on the same seeds, the valid randomisation is a paired sign-flip over the blocks, and at five blocks its smallest two-sided p is $2/2^5 = 0.0625$. Rather than report the programme’s central channel result as “directional” indefinitely, or add seeds until something crossed a threshold, we sealed an amendment before generating a single new token. It fixes *one* confirmatory test — the typed contract against baseline on `pass@10`, the paper’s declared primary endpoint — one fresh seed (47), and a hard stop: no seed beyond 47, whatever the result. Constants was excluded by that amendment on identifiability grounds, its five-seed difference vector containing zero blocks that no sixth block can make significant. The amendment also records the two operational deviations forced by a smaller GPU — a generation batch reduced from ten to five, applied uniformly, and a corresponding fix to a chunked-sampling defect in the harness — both fixed before any seed-47 output was scored, so seed 47 is one well-defined block rather than a moving target. The regenerated tokenizer was accepted only after hashing to the sealed identity byte-for-byte.

Seed 47 returned a typed–baseline `pass@10` difference of $+6$ tasks, the sixth positive block of six (3, 5, 4, 5, 9, 6); the

TABLE X

CANDIDATE-LEVEL RATES, LEAKAGE-CLEAN 1,740 CANDIDATES PER ARM, SEED 42. CONFORMANCE SATURATES WHILE CORRECTNESS STAYS AT BASELINE LEVEL.

input	compiled	passed
baseline	33.2%	0.52%
+ typed contract	76.0%	1.15%
typed contract, no binary	89.0%	0.57%

exact paired test gives $p=2/2^6 = 0.031$. The result is reported exactly as registered, and the two compilation contrasts that are also sign-consistent over six blocks are reported beside it as secondary descriptives, not as further confirmatory tests.

D. Withholding the binary entirely

The three arms above perturb the binary. The limiting case removes it. We pre-registered a control — hypotheses, thresholds, contrasts and input hashes sealed before any generation — that preserves the typed contract byte-for-byte and replaces the entire binary payload with the empty string, task-invariant, at the same frozen checkpoint.

We predicted `compile@10` $\in [160, 172]$ and `pass@10` $\in [1, 3]$. Both predictions were wrong, in the same direction: compilation came in at 174 and 173 of the leakage-clean 174 (the prediction was registered on the full 175), and correctness at 5 and 3. The registered branch that fired is *substantial type- or benchmark-prior performance*, which obliges us to re-interpret the typed-contract result rather than report the control as a floor.

Conformance saturates; correctness does not move.

Table X shows it on a denominator ten times larger than the task-level one: deleting the binary and keeping the contract gives the highest compilation rate we measured while candidate-level correctness stays at baseline level (0.57% versus 0.52%) — an earlier draft called this arm the least correct, which the leakage-clean denominator does not support, and we withdraw that reading. Given what `compile@k` actually asks (Section III), the compilation half is close to definitional — the model was handed the declared signature, which is most of what the front end checks — and we claim nothing more from it than that. The informative half is that near-perfect interface conformance buys *nothing* in correctness: candidate-level correctness is statistically indistinguishable from baseline, and at task level this arm solves no more than the baseline that reads the binary without types, and fewer than half as many as the arm given both.

The outputs are wrong, not malformed. Mean distinct candidates per task are 9.95 and 9.97 of 10, so this is not decoding collapse. Every frequent failure diagnostic is a runtime assertion — mismatched list, map or set lengths, `false != true`, `RangeError` — rather than a static error. The programs compile against the real tests, execute, return values of the declared type, and compute the wrong thing.

What the control does not license. Against the same-seed typed+F2 arm, the correctness deficit is directionally consistent

TABLE XI

DESCRIPTIVE CROSS-SYSTEM REFERENCE POINTS, FULL 175-TASK RUNS PREDATING THE SINGLE-TASK EXCLUSION, $k=10$. THE ORDERING IS CONSISTENT WITH THE CHANNEL EXPERIMENTS; THE TABLE ITSELF, WHICH MIXES MODEL, TRAINING, INPUT AND SCALE, SUPPORTS NO CAUSAL CLAIM.

system	input	pass@10	compile@10
Decoder-only 8B	assembly only	1	103
Decoder-only 8B	+ recovered consts.	4	107
Frontier reasoning	+ recovered consts.	4	86
Decoder-only 8B	+ oracle consts.	10	121
Encoder-decoder	assembly + CFG	7	124
Encoder-decoder	+ typed contract	14	171

in both seeds and significant in one (3 versus 8 discordant, $p=0.227$; 1 versus 10, $p=0.012$); our pre-registration required significance in both, so we do not claim a quantified F2 correctness contribution from this control. With types held constant, removing the binary gains 5 compiling tasks and loses 0 and 1 respectively ($p=0.0625$, which is the *minimum attainable* p at five discordant pairs, and $p=0.219$), so the compilation direction replicates but neither seed resolves it. The arm is a gold-derived oracle: it assumes perfect type and arity recovery while evaluating no recovery front end, and the empty input is out of distribution for this policy. We make no equivalence claim against baseline in either metric, and the 4.0 correctness figure is *baseline-scale*, not *baseline-equivalent* — the two arms' solved sets overlap in one task.

E. One benchmark, four systems

Table XI places every system we measured on the same 175 name-scrubbed tasks. Every row was re-derived from the raw generation artifacts under two toolchain revisions, returning identical counts.

A frontier reasoning model and a fine-tuned 8B decoder given the same recovered constants both solve four tasks. We present that as a coincidence of magnitude rather than an established equivalence: the solved sets were not compared, both intervals are wide, and Section V shows that at these rates two runs of a *single* system already disagree about most of what either solves, so equal counts are especially weak evidence of equal behaviour. The defensible reading of the table is the column ordering — `pass@10` moves with what the input contains, and barely with what consumes it.

F. Objective-side interventions do not substitute

If the deficit were a training-signal problem, objective-side work should close it. Reinforcement learning never reached a step (seven launches, hard OOM at ~ 94 GiB). Dense knowledge distillation is not constructible from a top- k API. Expanded synthetic SFT changed `pass@10` by $+0.0065$ $[-0.020, +0.039]$.

Two further programmes are often described as null results and should not be: reinforcement learning never completed a training step (seven launches, hard OOM at ~ 94 GiB) and dense distillation was not constructible from a top- k API. Those

are feasibility failures, not measurements, and we withdraw them from the paper’s negative findings.

Rejection-sampled fine-tuning we pursued to exhaustion. Across six checkpoints, three arrangements (a single pass, sequentially stacked passes, and one folded pass over the union of all harvested targets) and two replay conditions, `pass@10` ranged from 12 to 18 against a baseline of 14 — every value inside the floor. One arrangement did produce a measurable effect, in the wrong direction: stacking a second pass on the first left `compile@10` and `pass@1` unchanged or higher while `pass@10` fell from 18 to 12, with mean distinct candidates per task falling from 10.00 to 9.45. The policy became more reliable on what it already produced and lost the tail draws that `pass@k` depends on. Folding the same data into a single pass from the supervised checkpoint removed the collapse (9.98) and produced no gain.

Against all of this, the information-side lever works. Prepending constants recovered from the binary raises `pass@10` from 0.57% to 5.71% with `compile@10` at 69.1%. We state it carefully: that arm reconstructs constants by regex over *gold source* and is an oracle upper bound establishing the bottleneck is *relievable*, not a deployable system. Our binary-grounded extractor reaches 2.29%, succeeding on 1,746 of 1,755 tasks at string recall 0.465 micro against its own ≥ 0.8 acceptance bar.

G. Cross-ISA observation, and two reasons not to lean on it

On 343 held-out functions from real obfuscated Flutter release APKs targeting ARM64 — production binaries rather than a HumanEval port — `pass@10` is 5.54% (19 tasks) and `compile@5` is 93.07%. Pre-registered length bins fall monotonically to zero, and of 168 tasks with ≥ 200 instructions none was solved, while `compile@1` decays only gently ($0.65 \rightarrow 0.39$). The model keeps producing compilable code and stops producing correct code, which is the same qualitative pattern the x86 decomposition resolves into channels.

We give two reasons, both drawn from this paper’s own standards, for reporting that number as an observation rather than a result. First, the corpus retains original function names and the run supplied the full semantic signature, so 5.54% is measured under exactly the condition Section VI criticises. Second, it is a *single seed*. By Section V a lone draw at a 5.5% success rate fixes neither the count nor the identity of the 19 solved tasks, and we have not replicated it. Either defect alone would disqualify the figure as evidence about the ISA; together they make it a length-scaling observation and nothing more.

We state what the replication now requires, because it is unusually cheap and the instrument is unusually clean. The frozen checkpoint is recoverable and the four signature views are built and validated (343/343 rows, zero residual names, assembly byte-identical to the unmodified arm), and the signature mode and a binary-channel ablation are both existing configuration flags, so the study is inference-only with no code change. More important than cost, that architecture separates the two channels *physically*: the signature is tokenised text

while the binary enters as a learned soft prefix of embeddings, so unlike the single serialised prompt used in Sections VI and X, leakage between the channels is structurally impossible and zeroing the binary is exact.

One limit is worth stating in advance. Baseline compilation on that corpus is 339/343, so `compile@k` has four tasks of upward headroom and interventions that *raise* compilation — including the effect of Section X-D — cannot be reproduced there. The correctness half of the decomposition is testable; the compilation half is testable only downward.

XI. A POST-MORTEM ON EVALUATION LEAKAGE

An internal audit found that our historical prompt builder extracted up to ten assertion lines from each benchmark row’s `tests` field into the policy prompt; the evaluator then appended the same field to the generated code and executed it. All 154 rows leaked, 38 saturated the ten-line cap, and 133 of 154 tasks (86%) had their entire suite inside the window. Truncation preserved the fixed prefix and trimmed *assembly*, so the leak survived precisely where the binary evidence did not.

Every archived functional pool affected has been withdrawn. Because no historical run summary pinned a prompt hash, source commit or dataset hash, and file timestamps are not proof of provenance, the exclusion was applied as a blanket rule — covering pools that may well be clean — with the affected tables retained as audit records rather than as results. The corpora underlying the prior studies of Section VIII carry no scoring-test field and are not affected by *this specific* leak; that phrasing is deliberate, as it is not a clean bill of health. All results in Sections VI and VII postdate the fix and run under the schema `antigravity-v2-no-test-hints` with `rows_leaking_scoring_tests: 0`.

XII. RECOMMENDATIONS

- 1) **Report an identifier-scrubbed arm.** Scrub the function name, the parameter names *and the symbol in the disassembly*; a benchmark that supplies any of them cannot separate decompilation from reading a specification. On our data the two differ by a factor of six, and the disassembly symbol is the channel through which published C evaluations actually leak it (Table I). None of the six configurations we surveyed reports such an arm. This applies with particular force to HumanEval ports, whose reference solutions are near-certainly in pretraining data.
- 2) **Include a signature-only baseline that never sees the binary.** It costs *no* training run — the untrained base model suffices, and in our study it was not beaten — and it is the cheapest way to show the binary contributes.
- 3) **Report the sign stability of effects across seeds**, not only confidence intervals. A reversing sign and a wide interval are different findings.
- 4) **Report tasks solved in every seed and in any seed beside the mean.** Two integers, and they expose the gap between a reproducible score and a reproducible capability (Section V).

Withhold any claim about *which* tasks a method fixes until that gap is shown to be small.

- 5) **Decompose best-of- k surface metrics.** A rise in a max-over- k score computed only on compiling candidates may be a coverage effect; report the compiled-only mean beside it.
- 6) **Fail closed on test content in prompts,** and record a prompt-schema string in provenance.
- 7) **Pin artifacts and refuse partial scores.** A budget-truncated denominator is not a random sample.

XIII. THREATS TO VALIDITY

Construct. That the benchmark measures its own identifiers rather than decompilation is the paper’s subject rather than a caveat to it. Two limits on that claim are ours to state. First, the ablation removes a *bundle* — function name, parameter names, disassembly symbol — so it licenses no attribution to any one of them; the parameter names alone are strong task cues on HumanEval. Second, a descriptive identifier is simultaneously a retrieval key and a specification, and no arm here separates memorisation from specification-driven inference. The discriminating designs are paraphrased and misleading identifiers, and tasks authored after the model’s data cutoff; we have run none of them. Our CODEBLEU implementation uses a node-type multiset overlap and an identifier-level dataflow heuristic.

Internal. Design A is an inference-contract ablation on a frozen checkpoint whose training corpus had no signature field, so the comparator prompt was itself off-distribution; the effect size is far larger than that perturbation plausibly explains, but the exact figures are conditional on that checkpoint. The component isolations in Table VIII are single-seed diagnostics. The studies summarised in Section VIII are unreplicated.

External. One source language, two ISAs, one decoder family. The frontier measurement rests on one provider’s model on one date. Our survey (Section I-A) revises what we previously believed about transfer: HumanEval-Decompile is clean, but the same system’s real-world splits ship the true symbol in the assembly, so the critique *does* reach C evaluation practice — through a channel we had not considered until the survey found it. What remains open is whether the *channel decomposition* reproduces on another language and instruction set. Section X-G sets out exactly what that costs on our ARM64 corpus — inference alone on a frozen checkpoint, with the views already built and the channels physically separated — along with the one cell it cannot test. We regard it as the most valuable next study rather than a limitation we can argue away. We note that it would also vary the model architecture and the input representation alongside the instruction set, which forecloses any comparison of absolute magnitudes between the two corpora but makes a surviving dissociation a conceptual replication rather than a retest.

Conclusion. Several nulls are underpowered by construction, and where discordant counts are small we report the minimum attainable p so a null is not mistaken for evidence of absence. We apply Section V by discordance: aggregate comparisons and

the 35-to-0 identifier containment stand on their exact p -values, while task-level claims resting on a handful of discordant pairs — the solved-set overlaps of Section X-D — we decline to make.

XIV. CONCLUSION

Two experiments, one an inference-time ablation and one an untrained baseline, show that HumanEval-derived decompilation benchmarks substantially measure their own semantic identifiers: removing the function name, the parameter names and the disassembly symbol — while holding types, arity and the instruction stream fixed — collapses functional correctness by a factor of six while *improving* compilation, and an untrained baseline given the signature alone is not measurably worse than models fine-tuned on binaries. Exploratory architecture, fine-tuning and objective-side sweeps move nothing beyond the design’s resolution. Against that, one channel effect — the typed contract on functional correctness — is certified at $p=0.031$ under a pre-registered six-seed test with a hard stop.

Those identifiers are a specification. Whether the model recalls a memorised solution or reads the specification and solves the task afresh, our design cannot say, and we have stopped claiming it can. Either way the benchmark cannot attribute the score to the binary.

Underneath those results is the measurement problem of Section V: at these success rates a `pass@k` score replicates to one task while the tasks behind it turn over almost completely, so the number certifies an aggregate, not a capability on any particular function.

Any input channel teaches a small model the syntax and interface of a modern language; none of the channels we tried teaches its semantics until the input carries the information the compiler erased. The practical consequences are narrow and firm: *scrub the identifiers* — including the one in the *disassembly* — and *report that arm*, or the number is not a measurement of decompilation; and *report the tasks solved in every seed beside the mean*, or the number is not a measurement of a capability. Across six published evaluation configurations, no one currently does either.

DATA AVAILABILITY

The replication package — corpora and manifests, the object-pool extractor, the sealed frontier harness with its hashes, per-seed generations and scores, the pre-registration files, the contamination notice and the machine-readable integrity reports — is archived at <https://doi.org/10.5281/zenodo.21822791> with code under Apache-2.0 and documentation under CC-BY-4.0; raw benchmark-derived material remains under its upstream terms as recorded in the package’s licence map.

REFERENCES

- [1] H. Tan *et al.*, “LLM4Decompile: Decompiling binary code with large language models,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [2] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.

- [3] R. Abualazm and A. AboElHassan, “LLMs as idiomatic decompilers: Recovering high-level code from x86-64 assembly for Dart,” 2026, accepted at SANER 2026 ERA Track.
- [4] R. Abualazm, A. AboElHassan, and A. G. Wassal, “Evaluating fine-tuning and metrics for neural decompilation of Dart AOT binaries,” 2026, arXiv:2607.06125.
- [5] C. Cifuentes, “Reverse compilation techniques,” Ph.D. dissertation, Queensland University of Technology, 1994.
- [6] E. J. Schwartz, T. Avgerinos, and D. Brumley, “Native x86 decompilation using semantics-preserving structural analysis and iterative control-flow structuring,” in *USENIX Security Symposium*, 2013.
- [7] K. Yakdan, S. Eschweiler, E. Gerhards-Padilla, and M. Smith, “No more gotos: Decompilation using pattern-independent control-flow structuring and semantics-preserving transformations,” in *Network and Distributed System Security Symposium (NDSS)*, 2015.
- [8] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, “BAP: A binary analysis platform,” in *Computer Aided Verification (CAV)*, 2011.
- [9] D. S. Katz *et al.*, “Using recurrent neural networks for decompilation,” in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018.
- [10] X. Fu, R. Sun, P. Yang, and J. Xue, “CODA: An end-to-end neural program decompiler,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [11] P. Liu, J. Sun, L. Chen, Z. Yan, P. Zhang, D. Sun, D. Wang, and D. Li, “Control flow-augmented decompiler based on large language model,” 2025.
- [12] Y. G. Achamyeleh, H. Thomare, and M. A. A. Faruque, “HELIOS: Hierarchical graph abstraction for structure-aware LLM decompilation,” 2026.
- [13] Y. Wang, X. Xu, X. Zhu, X. Gu, and B. Shen, “SALT4Decompile: Inferring source-level abstract logic tree for LLM-based binary decompilation,” 2025.
- [14] H. Tan, W. Li, X. Tian, S. Wang, J. Liu, J. Li, and Y. Zhang, “SK2Decompile: LLM-based two-phase binary decompilation from skeleton to skin,” 2025.
- [15] N. Jiang, C. Tan, H. Ruan, W. Luo, X. Li, X. Zhang, and Q. Luo, “Nova: Generative language models for assembly code with hierarchical attention and contrastive learning,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [16] P. Liu, J. Sun, M. Xing, Y. Zeng, Z. Yan, L. Zhang, L. Chen, and D. Li, “Binary decompilation LLM with feedback-driven multi-turn refinement,” 2026.
- [17] L. Dramko, C. L. Goues, and E. J. Schwartz, “Neural decompilation with joint code and type prediction,” 2025.
- [18] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, “CodeBLEU: A method for automatic evaluation of code synthesis,” 2020.
- [19] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, and A. Jangda, “MultiPL-E: A scalable and extensible approach to benchmarking neural code generation,” *IEEE Transactions on Software Engineering*, vol. 49, no. 7, pp. 3675–3691, 2023.
- [20] Z. Gao, Y. Cui, H. Wang, S. Qin, Y. Wang, B. Zhang, and C. Zhang, “DecompileBench: A comprehensive benchmark for evaluating decompilers in real-world scenarios,” in *Findings of the Association for Computational Linguistics: ACL*, 2025.
- [21] W. Wangwarunyoo, “B(l)utter: Flutter mobile application reverse engineering tool,” <https://github.com/worawit/blutter>, 2023.
- [22] PT Security, “reflutter: Flutter reverse engineering framework,” <https://github.com/Impact-I/reFlutter>, 2021.
- [23] PNF Software, “Dart AOT snapshot helper plugin for JEB,” <https://www.pnfsoftware.com/blog/>, 2022, dart AOT snapshot helper plugin announcement.